Final Team Project: Multi-Agent Financial Analysis System

Group 3

Tushar Kumar, Juan Norena, and Emmanuel Sadek

Shiley-Marcos School of Engineering, University of San Diego

AAI-520-02: Natural Language Processing and GenAI

Mirsardar Esmaeili, PhD

October 19, 2026

AI Contribution Statement

We used two generative artificial intelligence (AI) tools in this project. When building the code, we used ChatGPT (OpenAI, 2026) to assist with debugging, syntax clarification, and outlining initial code structures, which we then reviewed, modified, and integrated ourselves. On October 19, 2026, we used Claude to check our draft against our project code and results. It suggested corrections to the descriptions of several tools and functions and to the reported scores, and those corrections are included in this version.

We then asked Claude to "use APA style guide for the report and also APA cite." In response, it formatted the report in APA Style, made small wording and punctuation edits to follow APA guidelines, suggested the published sources cited in the report, and drafted the introduction, the sentences in the System Design section that connect our design to those sources, the reference list, and this statement. Last, we asked Claude to check the report against our Jupyter notebook. It corrected small details in the System Design and Results sections and rewrote the Conclusion and Limitations from the findings and caveats recorded in the notebook. We reviewed all AI-generated content, checked each cited source, and take full responsibility for the accuracy of this report. All final implementation, analysis, and decisions are our own.

Final Team Project: Multi-Agent Financial Analysis System

This report describes the Stock Research Agent, a multi-agent financial analysis system that our team built for the final project. Given a stock ticker, the system plans which data to collect and gathers price history, financial statements, earnings data, and recent news. Large language model (LLM) agents then analyze the data and write a short research report. The design follows three workflow patterns for LLM agents: prompt chaining, routing, and evaluator–optimizer (Schluntz & Zhang, 2024). The system also grades and revises its own drafts, and it saves lessons from earlier runs so that later reports avoid the same gaps. The grader uses GPT-4o (OpenAI, 2024a), and every other model calls GPT-4o Mini (OpenAI, 2024b). The following sections explain each part of the system and then present our results for Apple and a memory test with three other companies.

System Design

Data Collection and Preprocessing

We use `get_news` to find the company's full name through Yahoo Finance (https://finance.yahoo.com/) and search for related news. This is useful because most articles refer to companies by name instead of tickers. The search includes the company name or ticker with words such as *earnings*, *revenue*, and *stock*. This helps us narrow down the search and cut useless articles. The News API returns up to 10 English-language articles from the last 21 days (News API, n.d.). We combine each article's title and description into one text entry. The `preprocess()` function removes extra spaces, duplicate entries, and articles marked `[Removed]`. In doing this, the system gets cleaner information to work with.

Metrics

The `get_prices` and `get_financials` tools get stock prices, annual financial statements, and the latest quarterly balance sheet from Yahoo Finance. All the numbers are calculated in code, so our model only interprets the results. The next step is to remove missing closing prices and use `pct_change()` to calculate daily returns. This is because today's row can be empty while the market is open. The code then calculates the overall price return, annualized volatility, and maximum drawdown. These show how much the price fluctuated and the largest decline from a peak during the selected period.

We also calculate revenue growth, net profit margin, debt-to-equity, and the current ratio. These help describe the company's profitability, debt, growth, and ability to cover short-term liabilities. For more clarity, we convert revenue, net income, and free cash flow into billions. The `get_valuation` tool returns the trailing and forward price-to-earnings (P/E) ratios, and `get_earnings` compares the last quarter's earnings per share (EPS) with the analyst estimate. Each tool returns its own dictionary, and `run_tool()` skips any tool that fails so the run can continue.

Research Planning

Our planner helps us determine which data tools each ticker needs and how much of the price history to pull out. The `planner_agent()` function gives the model the ticker, the company's sector and industry, a short description of each tool, and lessons saved from earlier runs. It gets the model to select the three tools that matter most out of prices, financials, valuation, and earnings. The model then chooses six months, one year, or two years of price history. The response contains three JavaScript Object Notation (JSON) fields: `steps`, `period`,

and `reason`. The coordinator uses this response to decide which tools run and how much price history to retrieve. The saved lessons get the model to see details that earlier reports missed.

Article Agent and Prompt Chaining

The `article_agent()` function organizes each article before it gets sent to the specialist analysts. First, `classify_article()` reads the first 600 characters and labels the article as *earnings*, *market*, *news*, or *irrelevant*. If the article is relevant, `extract_facts()` pulls out the main facts for its category. Then, the `summarize_facts()` function turns those details into a one-sentence summary.

This process uses prompt chaining because the functions work in sequence (Schluntz & Zhang, 2024). They are passing information from step to step. Its main purpose is to turn article text into a category and a short summary that the system can route to the appropriate analyst.

Specialist Agents, Routing, and Self-Review

The `SPECIALISTS` dictionary defines each analyst's focus and the article categories it handles. The `route_articles()` function creates a list for each analyst and assigns articles based on their category. Earnings articles go to the financial analyst, market articles go to the market analyst, company news goes to the news analyst, and irrelevant articles get dropped. This step follows the routing pattern, in which an input is classified and then sent to a specialized task (Schluntz & Zhang, 2024).

The `analyst_agent()` function gives the analysts relevant metrics and up to five article summaries. The market analyst looks at returns, volatility, drawdown, and valuation. The financial analyst examines growth, profitability, debt, liquidity, cash flow, and the last earnings surprise. Our news analyst considers how events could affect revenue, sentiment, or risk.

Each analyst produces two to three findings using saved lessons about information missed in previous reports. The self-review happens later, after the writer turns these findings into a report. The evaluator grades the draft, and the optimizer fixes what failed.

Writing the Report

The `writer_agent()` function turns the analyst findings into the first draft of the report. It creates five sections: overview, price behavior, financial health, recent news, and conclusion. Our prompt instructs the model to use only numbers from the supplied data. This is important because the evaluator checks the numbers in the report against the data. The report is then returned as a string under the `report` key in a JSON response.

Memory Through Runs

The memory functions help the system avoid mistakes we found in earlier reports. After each run, the `save_lessons()` function records the items that the first draft missed in `agent_memory.json`. It skips duplicates and saves the exact text of each failed check. Because the checks are not tied to one company, what it learned can apply to future examples.

Then, the `load_lessons()` function retrieves these reminders. They are included in the planner, analyst, and writer prompts so the model knows what was left out previously. The model is not retrained, and it gets feedback on lessons missed earlier.

Evaluation Checklist

The checklist is built inside `run_research()`. It adds 15 general checks and one check for each selected metric, except dates and raw figures like revenue. The checks ask the report to explain risks and give a recommendation that has a confidence level and time frame. They also require it to acknowledge missing information and use certain words, such as *buy*, *hold*, or *sell*. This is because these word checks are done in code, and looser wording would not match. The

score we get is the percentage of checks passed, which tells us how well the report meets these requirements.

Evaluator and Optimizer

The `evaluator_agent()` function helps us check each of the requirements. Most checks are separate LLM calls, which require the model to quote the supporting sentences word for word before returning true or false. The rest are checked in code, such as the 400-word limit and a numerical check that compares dollar amounts, percentages, and decimal values in the report with numbers in the data. However, a matching number can still be used in the wrong context because the check only confirms that each number shows up in the data.

The `optimizer_agent()` function receives the report, failed checks, and supporting data. It revises the report and lists the changes made. The revised report is evaluated again and kept only if the score improves. This design follows the evaluator–optimizer pattern (Schluntz & Zhang, 2024). This feedback loop helps fix the missing requirements, but an evaluator can still miss some mistakes.

Full Workflow

The `run_research()` function is what we used to connect all our steps. It loads lessons, requests a plan, gets data, routes articles, runs the analysts that have tool data or articles, and generates a report. It then builds our checklist and starts doing the evaluation.

The system allows a maximum of three revisions and stops early if the score is 100. The revisions that do not improve the score are discarded. We save our lessons from the first draft and return both reports, evaluations, and score history.

Results

Apple Results

Our Apple example clears the existing memory before running. It displays the original and optimized reports with the checklist results. Our first draft passed 12 of 24 checks and gave us a score of 50. After two revisions, we saw a jump in the scores to 88 and 96. A third revision scored 92 and was discarded. The final report only failed the 400-word limit because the revisions made it longer.

The revisions added explanations of drawdown, free cash flow, and the earnings surprise; a recommendation; a confidence level; and missing-information limitations. This shows improved checklist coverage, but a high score does not guarantee that every claim is correct.

Memory Test

Microsoft, Johnson & Johnson, and Nvidia were each tested without and with memory. Both runs for each company reused the same plan, data, and article summaries, so the lessons only reached the analyst and writer prompts. The comparison focused on first drafts. Microsoft improved from 58 to 71, Johnson & Johnson from 62 to 92, and Nvidia from 67 to 88. The average increased from approximately 62 to 84.

The results suggest that saved reminders helped reports meet more requirements. Across the three companies, 19 checks that failed in a no-memory first draft passed in the memory first draft, and 12 of these were the same four checks: stating the fiscal year; using *buy*, *hold*, or *sell*; giving a confidence level; and naming something to watch. All four were Apple lessons that failed in every no-memory first draft. Memory-off runs did not save lessons, so each memory run only used feedback from other companies. However, lessons accumulate across runs, so Johnson & Johnson and Nvidia used 17 lessons, whereas Microsoft used 12. With only one test per

condition and variation in model responses and grading, memory cannot be treated as the only cause of improvement.

Conclusion and Limitations

The system combines specialized analysis, self-review, feedback, and memory to produce and improve stock research reports. The evaluator–optimizer loop raised the score in every run. Apple went from 50 to 96, and all seven runs (Apple plus the six memory-test runs) finished between 88 and 100. The memory runs had better first drafts, with the average score rising from 62 to 84. Most of this gain came from four checks that the Apple run had failed. Our revisions made the reports longer. First drafts ran 243 to 390 words, but four of the seven final reports went over the 400-word limit, even after rounds where the optimizer said it had cut the report.

These results have limits. Because saved lessons reflect the checklist, higher scores mainly show better coverage of its requirements, and with one run per condition we cannot separate the effect of memory from normal variation. The grader never sees the data, and the number check only confirms that each number appears somewhere in the data. For example, Johnson & Johnson's final no-memory report scored 92 while describing a 35.2% return as "significantly outperforming the S&P 500, which returned 52%." Every final report is named an "upcoming" earnings report on a date that had already passed when we ran the notebook, such as June 30, 2026, for Apple. The grader was also inconsistent. It passed a Johnson & Johnson sentence about maximum drawdown in the first draft and then failed the same sentence after the first revision. On the data side, article previews can omit details, missing financial fields can cause a whole tool to be skipped, and the planner picked the same three tools and a one-year window for every ticker. No headline was ever routed to the financial analyst, and Microsoft had

almost no news: only one of its nine headlines was classified as relevant, so its news analyst never ran. The reports still need human review to verify their evidence and conclusions.

References

News API. (n.d.). *Endpoints*. Retrieved October 19, 2026, from https://newsapi.org/docs/endpoints

OpenAI. (2024a). *GPT-4o* [Large language model]. https://developers.openai.com/api/docs/models/gpt-4o

OpenAI. (2024b). *GPT-4o Mini* [Large language model]. https://developers.openai.com/api/docs/models/gpt-4o-mini

OpenAI. (2026). *ChatGPT* [Large language model]. https://chatgpt.com/

Schluntz, E., & Zhang, B. (2024, December 19). *Building effective agents*. Anthropic. https://www.anthropic.com/engineering/building-effective-agents

AI Disclosure

AI tools were used to assist with debugging, syntax clarification, and outlining initial code structures that were then reviewed, modified, and integrated by ourselves. All final implementation, analysis, and decisions are our own work and reflect our understanding of the material.